

UT2: Manejo de Conectores (JDBC y Oracle Database)

Manual Técnico y Didáctico Completo: Arquitectura de Conexión, Consultas Seguras, Transacciones ACID, Procedimientos PL/SQL y Patrones de Persistencia

Módulo: Acceso a Datos (0486)

Ciclo: DAM · 2º Curso

Unidad Didáctica: UD02 (35 horas)

Resultados Aprendizaje: RA2 (CE2.a - CE2.e)

Motor de Referencia: Oracle Database (19c/21c/23c)

Docente: Sergio Capdevila Díez

1. Introducción y Contexto Profesional

En los sistemas empresariales modernos, las bases de datos relacionales continúan custodiando el núcleo transaccional más valioso de las organizaciones: cuentas bancarias, historiales clínicos, stocks de almacén y pedidos de comercio electrónico. En el ecosistema Java, la tecnología estándar y universal que permite a cualquier aplicación interactuar programáticamente con un sistema gestor relacional se denomina **JDBC (Java Database Connectivity)**.

JDBC actúa como una **capa de abstracción estandarizada** ubicada entre el código fuente Java y el motor de base de datos. Gracias a JDBC, un desarrollador utiliza exactamente las mismas clases e interfaces (`Connection`, `PreparedStatement`, `ResultSet`) para enviar sentencias SQL tanto si el motor subyacente es **Oracle Database**, PostgreSQL, MySQL o Microsoft SQL Server. La única pieza que varía es el **driver JDBC** específico suministrado por el fabricante del motor.

A lo largo de esta unidad trabajaremos sobre el motor empresarial por excelencia: **Oracle Database**. Aprenderás a configurar sus dependencias Maven, gestionar cadenas de conexión thin, ejecutar operaciones CRUD blindadas contra inyecciones SQL, controlar transacciones atómicas manuales con puntos de restauración (`Savepoint`), invocar lógica de negocio en PL/SQL mediante `CallableStatement` y estructurar tus aplicaciones bajo los patrones de arquitectura limpia más respetados en la industria: **Singleton**, **DTO (Data Transfer Object)** y **DAO (Data Access Object)**, junto con pools de conexiones de alta velocidad como **HikariCP**.

DEPENDENCIA MAVEN DEL DRIVER OFICIAL DE ORACLE ('POM.XML')

Para conectar tu proyecto Java 21 con Oracle Database 19c/21c/23c, añade el artefacto oficial a tu archivo

`pom.xml` :

```
<dependency>
  <groupId>com.oracle.database.jdbc</groupId>
  <artifactId>ojdbc11</artifactId>
  <version>23.3.0.23.09</version>
</dependency>
```

2. Conceptos Fundamentales: La Analogía del Teléfono

Para asimilar el funcionamiento de JDBC antes de sumergirnos en la sintaxis formal, imaginemos el siguiente paralelismo con una llamada telefónica corporativa internacional:

📞 ANALOGÍA DIDÁCTICA: EL CIRCUITO DE COMUNICACIÓN JDBC

- **El Protocolo Telefónico (Driver):** Para que dos personas hablen, necesitan una red telefónica compatible con los estándares del país de destino. El driver de Oracle (`ojdbc11.jar`) es el traductor de red que enseña a Java a hablar el dialecto binario propio de Oracle (*TNS / Oracle Net Services*).
- **La Conexión Activa (Connection):** Es el descolgado y establecimiento de la llamada entre tu aplicación y el puerto 1521 del servidor Oracle. Mientras la llamada está abierta, se consume ancho de banda y memoria en ambos extremos.
- **El Mensaje o Petición (Statement / PreparedStatement):** Es la pregunta u orden concreta que transmites por el auricular (*"Dígame el saldo del cliente 402"* o *"Transfiera 500 euros"*).
- **La Libreta de Respuestas (ResultSet):** Cuando el interlocutor te responde con una lista de 50 nombres, no te los dice todos de golpe en un microsegundo. Te los dicta fila a fila; tú vas apuntando con el cursor (`rs.next()`) hasta que dice *"no hay más"* (fin del cursor).

Arquitectura en Capas de JDBC

La especificación JDBC se divide estrictamente en dos niveles:

1. **JDBC API (Nivel de Aplicación):** Definida en el paquete estándar `java.sql` del JDK. Contiene interfaces abstractas puras que no saben nada de ningún fabricante (`Connection` , `Statement` , `ResultSet`).
2. **JDBC Driver (Nivel de Implementación):** Proporcionado por Oracle. Son las clases concretas que implementan dichas interfaces (por ejemplo, `oracle.jdbc.driver.OracleConnection`) y se comunican a bajo nivel por TCP/IP mediante el protocolo *Thin*.

3. Desarrollo Exhaustivo y Fichas de Métodos de JDBC

A continuación se desglosan sistemáticamente todas las interfaces, clases y métodos fundamentales de la API JDBC. Cada instrucción se detalla con su impacto en bajo nivel, parámetros, retorno y código funcional.

3.1. Gestión de Conexiones: DriverManager y DataSource

DriverManager.getConnection(String url, String user, String password) JAVA.SQL.DRIVERMANAGER

Retorno: `Connection` activa.

Excepciones: `SQLException` si falla la red, puerto o credenciales.

¿Qué hace en bajo nivel? Abre un socket TCP/IP contra la dirección IP y puerto indicados en la URL (por defecto 1521 en Oracle). Realiza el handshake de red con el *Listener* de Oracle, envía el paquete de autenticación criptográfica con el usuario y contraseña, y fuerza al servidor Oracle a instanciar un proceso de servicio y reservar un segmento de memoria privada (PGA) para la sesión.

Formato de la URL Thin de Oracle: `jdbc:oracle:thin:@host:puerto:SID` o con Service Name: `jdbc:oracle:thin:@//host:puerto/service_name`.

```
String url = "jdbc:oracle:thin:@localhost:1521:xe";
String user = "c##dam2_user";
String pass = "oraclePass123";

try (Connection con = DriverManager.getConnection(url, user, pass)) {
    System.out.println("✅ Conexión física establecida con éxito: " + con.getMetaData().getDat
} catch (SQLException e) {
    System.err.printf("❌ Error de conexión [ORA-%05d]: %s%n", e.getErrorCode(), e.getMessage(
}
```

TRABAJO SÍNCRONO EN AULA: ACTIVIDAD 2.1

Abre tu IDE y resuelve la **Actividad 2.1 del Cuaderno de Actividades**: Configuración del entorno Maven con `ojdbc11` y prueba de conexión básica contra Oracle Database.

HikariDataSource: Pool de Conexiones de Alto Rendimiento

COM.ZAXXER.HIKARI.HIKARIDATASOURCE

Retorno: Instancia de `DataSource` pre-calentada.

Ventaja: Latencia cero al reutilizar sockets físicos ya abiertos.

¿Qué hace en bajo nivel? En lugar de abrir y destruir sockets en cada petición, HikariCP establece un conjunto fijo de conexiones físicas en segundo plano (por ejemplo, 10 sesiones permanentes en Oracle). Cuando el código solicita `dataSource.getConnection()`, el pool entrega en microsegundos una conexión libre. Al invocar `con.close()`, un proxy intercepta la llamada y devuelve la conexión al pool sin cerrar el socket físico.

```
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:oracle:thin:@localhost:1521:xe");
config.setUsername("c##dam2_user");
config.setPassword("oraclePass123");
config.setMaximumPoolSize(10);
config.setConnectionTimeout(10000); // 10 segundos espera máxima

HikariDataSource ds = new HikariDataSource(config);

// Préstamo ultrarrápido desde el pool
try (Connection con = ds.getConnection()) {
    // Operaciones JDBC...
} // Aquí con.close() devuelve la conexión al pool
```

3.2. Sentencias SQL Seguras: PreparedStatement vs Statement

⚠ EL PELIGRO LETAL DE LA INYECCIÓN SQL CON STATEMENT

El uso de `java.sql.Statement` concatenando cadenas de texto permite que un usuario malintencionado altere la lógica de la sentencia enviada al motor:

```
// CÓDIGO VULNERABLE (¡PROHIBIDO EN PRODUCCIÓN Y EXÁMENES!)
String sql = "SELECT * FROM usuarios WHERE user = '" + inputUser + "' AND pass = '" + inputPass
Statement st = con.createStatement();
ResultSet rs = st.executeQuery(sql);
```

Si el atacante introduce como usuario `admin' --` o `' OR '1'='1`, la consulta resultante anula la comprobación de la contraseña y permite el acceso ilegítimo a cualquier cuenta.

Retorno: Objeto `PreparedStatement` precompilado. **Seguridad:** Inmunidad matemática absoluta frente a Inyección SQL.

¿Por qué es seguro en bajo nivel? La sentencia SQL con interrogaciones `?` (parámetros posicionales) se envía al motor Oracle **antes** de recibir los datos. El optimizador de Oracle compila la consulta, crea el árbol sintáctico y define el plan de ejecución en la memoria compartida (SGA). Cuando posteriormente se asignan los valores con `setString()`, Oracle los trata estrictamente como datos literales puros, siendo totalmente imposible que un carácter especial como comillas o guiones altere la estructura de la consulta.

Fichas de Asignación de Parámetros (Índices basados en 1):

- `ps.setInt(int parameterIndex, int x)` : Asigna un entero de 32 bits.
- `ps.setString(int parameterIndex, String x)` : Asigna texto con escape automático de caracteres.
- `ps.setDouble(int parameterIndex, double x)` : Asigna un decimal de coma flotante.
- `ps.setDate(int parameterIndex, java.sql.Date d)` : Asigna una fecha SQL (solo año, mes y día).
- `ps.setTimestamp(int parameterIndex, java.sql.Timestamp t)` : Asigna fecha y hora con precisión de milisegundos.
- `ps.setNull(int parameterIndex, int sqlType)` : Inserta explícitamente un valor `NULL` de `java.sql.Types`.

```
String sql = "INSERT INTO productos (id, nombre, precio, stock, fecha_alta) VALUES (?, ?, ?, ?, ?)";

try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setInt(1, 101);
    ps.setString(2, "Monitor Gaming 27\");
    ps.setDouble(3, 249.99);
    ps.setInt(4, 15);
    ps.setDate(5, java.sql.Date.valueOf("2026-09-17"));

    int filasAfectadas = ps.executeUpdate();
    System.out.println("Filas insertadas: " + filasAfectadas);
}
```

executeUpdate(): Para DML (INSERT, UPDATE, DELETE) y DDL. Retorna `int` (nº de filas afectadas). **executeQuery():** Exclusivo para SELECT. Retorna `ResultSet` con los datos.

Regla de oro: Nunca invoques `executeQuery()` para una operación de modificación (lanzará una excepción en la mayoría de drivers), ni uses `executeUpdate()` para una consulta SELECT.

TRABAJO SÍNCRONO EN AULA: ACTIVIDADES 2.2 Y 2.3

Realiza las **Actividades 2.2** (Demostración de ataque de Inyección SQL con Statement) y **2.3** (Operaciones CRUD seguras y parametrizadas con PreparedStatement) del Cuaderno de Aula.

3.3. Procesamiento de Resultados: La Interfaz ResultSet

rs.next() y Métodos de Extracción Tipada

JAVA.SQL.RESULTSET

rs.next(): Avanza el cursor a la siguiente fila. Retorna `true` si existe o `false` al terminar.

Posición Inicial: El cursor se sitúa inicialmente *antes de la primera fila* (Before First).

¿Cómo funciona el cursor? Un `ResultSet` mantiene un puntero en memoria que apunta a una fila activa a la vez. Es obligatorio llamar a `rs.next()` al menos una vez antes de intentar leer cualquier dato, de lo contrario la JVM lanzará `SQLException: Cursor fuera de rango`.

Lectura Tipada por Nombre de Columna vs por Índice:

- Por nombre (recomendado para mantenibilidad): `rs.getString("nombre")`, `rs.getDouble("precio")`.
- Por índice (más rápido en bucles críticos, base 1): `rs.getString(1)`, `rs.getDouble(2)`.
- **Detección de NULOS con `rs.isNull()`**: Los tipos primitivos Java (`int`, `double`, `boolean`) no pueden ser `null`. Si una columna numérica en Oracle contiene `NULL`, JDBC devuelve `0`. Para verificar si ese cero era un cero real o un nulo, debes comprobar inmediatamente: `if (rs.isNull()) { ... }`.

```
String sql = "SELECT id, nombre, salario, comision FROM empleados WHERE departamento = ?";

try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, "Ventas");
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            int id = rs.getInt("id");
            String nombre = rs.getString("nombre");
            double salario = rs.getDouble("salario");

            double comision = rs.getDouble("comision");
            Double comisionObj = rs.isNull() ? null : comision;

            System.out.printf("Empleado #d: %s | Salario: %.2f € | Comisión: %s\n",
                id, nombre, salario, (comisionObj != null ? comisionObj + " €" : "Sin comi"));
        }
    }
}
```

3.4. Transacciones ACID y Puntos de Restauración (Savepoint)

Por defecto, las conexiones JDBC operan con **Auto-Commit activado** (cada `executeUpdate` se graba de forma irreversible en disco inmediatamente). Para garantizar la integridad en transferencias o ventas compuestas, debemos gobernar la transacción manualmente.

con.setAutoCommit(false), commit(), rollback() y setSavepoint()

JAVA.SQL.CONNECTION /
SAVEPOINT

commit(): Consolida permanentemente todas las operaciones pendientes en el disco de Oracle.

rollback(): Deshace todos los cambios realizados desde el inicio de la transacción o hasta un Savepoint.

Protocolo Transaccional Obligatorio:

1. Desactivar confirmación automática: `con.setAutoCommit(false);`.
2. Ejecutar sentencias DML correlativas dentro de un bloque `try`.
3. Si todo concluye satisfactoriamente: `con.commit();`.
4. En caso de cualquier error (bloque `catch (SQLException e)`): `con.rollback();`.
5. En el bloque `finally`: restaurar `con.setAutoCommit(true);` antes de cerrar o devolver al pool.

```
con.setAutoCommit(false); // 1. Iniciar transacción manual
Savepoint sp = null;

try {
    // Operación 1: Descontar stock
    actualizarStock(con, idProducto, cantidad);

    // Guardamos un punto de restauración parcial antes del módulo de fidelización
    sp = con.setSavepoint("PuntoStockConfirmado");

    try {
        // Operación 2: Asignar puntos de fidelidad (opcional)
        sumarPuntosFidelidad(con, idCliente, 50);
    } catch (SQLException eFidelidad) {
        // Si falla la fidelidad, revertimos solo esa parte sin anular el pedido principal
        con.rollback(sp);
        System.out.println("⚠ No se pudieron asignar los puntos, pero el pedido continúa.");
    }

    con.commit(); // 2. Confirmar transacción completa
    System.out.println("✅ Transacción confirmada en Oracle con éxito (COMMIT).");
} catch (SQLException e) {
    con.rollback(); // 3. Reversión total ante fallo crítico
    System.err.println("❌ Error grave en pedido. Reversión total (ROLLBACK): " + e.getMessage());
} finally {
    con.setAutoCommit(true); // 4. Restaurar estado base
}
```

TRABAJO SÍNCRONO EN AULA: ACTIVIDADES 2.4 Y 2.5

Resuelve las **Actividades 2.4** (Control transaccional estricto con rollback) y **2.5** (Gestión de puntos de restauración parciales con Savepoint) en tu IDE.

3.5. Procedimientos y Funciones Almacenadas: CallableStatement

con.prepareCall(String callSql) y registerOutParameter()

JAVA.SQL.CALLABLESTATEMENT

Sintaxis Procedimiento: "{call NOMBRE_PROC(?, ?, ?)}"

Sintaxis Función: "{? = call NOMBRE_FUNC(?)}"

¿Qué hace en bajo nivel? Ejecuta código PL/SQL compilado directamente en el motor Oracle. Los parámetros de salida (OUT) deben registrarse con su tipo SQL mediante `cs.registerOutParameter(posicion, java.sql.Types...)` antes de ejecutar la llamada, y recuperarse tras `cs.execute()` con `cs.getString()`, `cs.getDouble()`, etc.

```
String call = "{call sp_calcular_descuento(?, ?, ?)}";

try (CallableStatement cs = con.prepareCall(call)) {
    // Parámetros de entrada (IN)
    cs.setInt(1, 105);           // ID Cliente
    cs.setDouble(2, 500.0);      // Importe compra

    // Parámetro de salida (OUT)
    cs.registerOutParameter(3, java.sql.Types.NUMERIC);

    // Ejecución en el servidor Oracle
    cs.execute();

    // Lectura del valor devuelto por PL/SQL
    double descuentoFinal = cs.getDouble(3);
    System.out.printf("Importe final con descuento aplicado por Oracle: %.2f €\n", descuentoFinal);
}
```

3.6. Inspección Dinámica del Esquema: DatabaseMetaData y ResultSetMetaData

ResultSetMetaData: Inspección de Columnas Dinámicas

JAVA.SQL.RESULTSETMETADATA

Obtención: `ResultSetMetaData rsmd = rs.getMetaData();`

Uso: Descubrimiento de estructura cuando la consulta no es conocida a priori.

Métodos Fundamentales:

- `rsmd.getColumnCount()` : Número total de columnas retornadas.
- `rsmd.getColumnLabel(int col)` : Nombre o alias de la columna (base 1).
- `rsmd.getColumnTypeName(int col)` : Tipo de dato SQL en Oracle (ej. `VARCHAR2`, `NUMBER`).
- `rsmd.isNullable(int col)` : Indica si la columna admite nulos (`columnNoNulls`, `columnNullable`).

```
ResultSetMetaData rsmd = rs.getMetaData();
int columnas = rsmd.getColumnCount();

for (int i = 1; i <= columnas; i++) {
    System.out.printf("Col %d: %-15s [Tipo: %-10s | Nulos: %s]%n",
        i, rsmd.getColumnLabel(i), rsmd.getColumnTypeName(i),
        (rsmd.isNullable(i) == ResultSetMetaData.columnNullable ? "SÍ" : "NO"));
}
```

TRABAJO SÍNCRONO EN AULA: ACTIVIDADES 2.6 Y 2.7

Desarrolla en clase las **Actividades 2.6** (Invocación de procedimientos PL/SQL con CallableStatement) y **2.7** (Constructor dinámico de tablas ASCII con ResultSetMetaData).

3.7. Arquitectura Profesional: Patrones Singleton, DTO y DAO

En proyectos profesionales nunca mezclamos consultas SQL directas con la lógica de negocio ni con las pantallas de usuario. Aplicamos el desacoplamiento en 3 capas:

Patrón DTO (Data Transfer Object) con Java Records

MODELO DE TRANSPORTE DESACOPLADO

Un DTO transporta datos limpios sin lógica de persistencia. En Java 21, la forma más limpia, inmutable y concisa de declarar un DTO es mediante **Java Records**:

```
// DTO inmutable con constructor compacto, getters (id(), nombre()), equals y toString automáticos
public record ProductoDTO(
    int id,
    String nombre,
    double precio,
    int stock,
    String categoria
) {}
```

Patrón DAO (Data Access Object): Interfaz Genérica e Implementación

CAPA DE ACCESO A
DATOS

El patrón DAO aísla completamente a la aplicación de la tecnología de base de datos. Se define una interfaz genérica que declara las operaciones CRUD y una implementación concreta para Oracle:

```
// 1. Interfaz Genérica DAO
public interface GenericDAO<T, ID> {
    void insertar(T entity) throws SQLException;
    Optional<T> buscarPorId(ID id) throws SQLException;
    List<T> listarTodos() throws SQLException;
    boolean actualizar(T entity) throws SQLException;
    boolean eliminar(ID id) throws SQLException;
}

// 2. Implementación Concreta para Oracle Database
public class ProductoDAOOracleImpl implements GenericDAO<ProductoDTO, Integer> {
    private final DataSource dataSource;

    public ProductoDAOOracleImpl(DataSource ds) {
        this.dataSource = ds;
    }

    @Override
    public Optional<ProductoDTO> buscarPorId(Integer id) throws SQLException {
        String sql = "SELECT id, nombre, precio, stock, categoria FROM productos WHERE id = ?";
        try (Connection con = dataSource.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return Optional.of(new ProductoDTO(
                        rs.getInt("id"),
                        rs.getString("nombre"),
                        rs.getDouble("precio"),
                        rs.getInt("stock"),
                        rs.getString("categoria")
                    ));
                }
            }
        }
        return Optional.empty();
    }

    // Resto de métodos CRUD...
}
```

4. Ejemplos Prácticos Graduados (Básico, Intermedio y Profesional)

4.1. Nivel Básico: Verificación de Conexión y SELECT DUAL en Oracle

```
// Prueba elemental de conectividad con Oracle
try (Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "system", "or
    try (Statement st = con.createStatement());
        ResultSet rs = st.executeQuery("SELECT SYSDATE, USER FROM DUAL")) {
        if (rs.next()) {
            System.out.printf("Servidor Oracle activo. Fecha: %s | Usuario conectado: %s\n",
                rs.getString(1), rs.getString(2));
        }
    }
}
```

4.2. Nivel Intermedio: Inserción por Lotes (Batch Processing) con PreparedStatement

Cuando se deben insertar miles de registros, enviar una petición de red por cada fila arruina el rendimiento. `addBatch()` acumula las órdenes en la memoria cliente y las envía a Oracle en un único viaje de red:

```
String sql = "INSERT INTO auditoria_accesos (id, usuario, ip, fecha) VALUES (?, ?, ?, ?)";

try (Connection con = ds.getConnection();
    PreparedStatement ps = con.prepareStatement(sql)) {

    con.setAutoCommit(false); // Optimización transaccional del lote

    for (int i = 1; i <= 1000; i++) {
        ps.setInt(1, i);
        ps.setString(2, "usuario_" + i);
        ps.setString(3, "192.168.1." + (i % 250));
        ps.setTimestamp(4, new java.sql.Timestamp(System.currentTimeMillis()));
        ps.addBatch(); // Encolar en el lote

        if (i % 200 == 0) { // Enviar en paquetes de 200 órdenes
            ps.executeBatch();
        }
    }
    ps.executeBatch(); // Enviar restos
    con.commit();
    System.out.println("✅ 1.000 registros insertados en Oracle mediante Batch en pocos milisegundos.
}
```

4.3. Nivel Profesional: Coordinación Transaccional en Capa de Servicio

En una arquitectura empresarial, dos DAOs distintos (ej. `CuentaDAO` y `MovimientoDAO`) deben compartir **la misma conexión física** durante una operación transaccional para que el `commit` o `rollback` afecte a ambos:

```

public class BancoService {
    private final DataSource dataSource;

    public BancoService(DataSource ds) { this.dataSource = ds; }

    public void transferirFondos(String ibanOrigen, String ibanDestino, double importe) throws SQLException {
        try (Connection con = dataSource.getConnection()) {
            con.setAutoCommit(false); // Inicia unidad de trabajo atómica

            CuentaDAO cuentaDAO = new CuentaDAOOracleImpl(con);
            MovimientoDAO movimientoDAO = new MovimientoDAOOracleImpl(con);

            try {
                // 1. Descontar saldo
                cuentaDAO.adeudar(ibanOrigen, importe);
                // 2. Abonar saldo
                cuentaDAO.abonar(ibanDestino, importe);
                // 3. Registrar auditoría contable
                movimientoDAO.registrar(new MovimientoDTO(ibanOrigen, ibanDestino, importe));

                con.commit(); // Confirmar los 3 pasos juntos
            } catch (SQLException e) {
                con.rollback(); // deshacer absolutamente todo si cualquier paso falla
                throw new ServiceException("Error en transferencia bancaria: " + e.getMessage(), e);
            } finally {
                con.setAutoCommit(true);
            }
        }
    }
}

```

5. Caso de Estudio Completo: Arquitectura DAO/DTO Bancaria

El siguiente esquema muestra el flujo completo de una consulta en tres capas desacopladas, utilizando HikariCP, PreparedStatement, DTO con Records y control de recursos:

```
// 1. DTO de Transferencia de Datos
public record CuentaBancaria(String iban, String titular, double saldo, boolean activa) {}

// 2. Implementación del DAO
public class CuentaBancariaDAO {
    private final DataSource ds;
    public CuentaBancariaDAO(DataSource ds) { this.ds = ds; }

    public boolean actualizarSaldo(String iban, double nuevoSaldo) throws SQLException {
        String sql = "UPDATE cuentas SET saldo = ? WHERE iban = ? AND activa = 1";
        try (Connection con = ds.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setDouble(1, nuevoSaldo);
            ps.setString(2, iban);
            return ps.executeUpdate() > 0;
        }
    }
}
```

6. Errores Habituales en JDBC y Códigos ORA en Oracle

Código / Excepción	Causa Profunda en Bajo Nivel	Cómo Detectarlo y Solucionarlo
ORA-01017: invalid username/password	Credenciales erróneas o cuenta bloqueada por intentos fallidos en Oracle. En Oracle 12c+ los usuarios locales deben prefijarse con <code>c##</code> .	Verificar el usuario en SQL*Plus con <code>ALTER USER c##usuario ACCOUNT UNLOCK;</code> y revisar la contraseña exacta.
ORA-00942: table or view does not exist	La tabla no existe en el esquema conectado o el usuario no tiene privilegios de lectura (<code>SELECT ON esquema.tabla</code>).	Prefijar el esquema (<code>SELECT * FROM c##admin.clientes</code>) o conceder permisos: <code>GRANT SELECT ON clientes TO c##usuario;</code> .
SQLTransientConnectionException (Connection leak)	El pool HikariCP se ha quedado sin conexiones disponibles porque se abrieron con <code>ds.getConnection()</code> sin bloque <code>try-with-resources</code> .	Activar en HikariCP <code>config.setLeakDetectionThreshold(5000);</code> para que imprima en consola qué línea de código olvidó cerrar la conexión.
SQLException: Result set after last row	Se intentó invocar un método de lectura (<code>rs.getString()</code>) tras haber finalizado el bucle <code>while (rs.next())</code> o sin haber llamado a <code>next()</code> previamente.	Asegurar que toda lectura de datos se encuentre dentro de un <code>if (rs.next())</code> o <code>while (rs.next())</code> .
SQLException: Exhausted Result Set	Típico error en Oracle al intentar leer de un cursor cerrado o vacío sin comprobar el retorno de <code>rs.next()</code> .	Comprobar siempre <code>if (rs.next())</code> antes de acceder a la primera columna.

7. Buenas Prácticas de Ingeniería de Software (Junior vs Senior)

Aspecto de Persistencia	Enfoque del Desarrollador Junior	Estándar del Desarrollador Senior / Arquitecto
Creación de Conexiones	Usa <code>DriverManager.getConnection()</code> en cada método, abriendo cientos de conexiones por minuto.	Configura un Pool de conexiones HikariCP dimensionado según la concurrencia y la memoria de Oracle.
Construcción de Consultas	Concatena variables Java con operadores <code>+</code> en objetos <code>Statement</code> .	Emplea exclusivamente PreparedStatement con parámetros posicionales <code>?</code> .
Cierre de Recursos	Cierra en el bloque <code>finally</code> comprobando nulos manualmente (propenso a leaks si <code>close()</code> lanza excepción).	Utiliza siempre bloques try-with-resources para cerrar automáticamente conexiones, sentencias y cursores.
Gestión Transaccional	Confía en el auto-commit por defecto; si una transferencia falla a la mitad, los datos quedan descompensados.	Desactiva el auto-commit (<code>false</code>), encierra las órdenes en <code>try-catch</code> y ejecuta <code>commit()</code> o <code>rollback()</code> explícitos.
Estructura del Proyecto	Escribe sentencias SQL dentro de la interfaz gráfica (Swing, JavaFX) o del controlador web.	Separa responsabilidades mediante DAO (Data Access Object) y DTO (Java Data Records) independientes.

8. Tablas Comparativas de Tecnologías JDBC

Comparativa: Tipos de Sentencias en JDBC

Característica	Statement	PreparedStatement	CallableStatement
Compilación en Oracle	Se compila en cada ejecución (Hard Parse lento)	Se compila una sola vez en la SGA (Soft Parse ultrarrápido)	Precompilada y almacenada en el motor (PL/SQL)
Seguridad	Vulnerable a Inyección SQL	Inmune a Inyección SQL	Inmune a Inyección SQL
Parámetros	Ninguno (solo texto plano)	Parámetros de entrada <code>?</code> (IN)	Parámetros <code>IN</code> , <code>OUT</code> e <code>IN OUT</code>
Caso de Uso	Scripts DDL únicos (CREATE, DROP)	El 95% de las consultas y modificaciones en Java	Procedimientos y funciones PL/SQL complejas

Comparativa: Modelado de Datos (POJO Clásico vs Java Record)

Criterio	POJO Tradicional (Java Beans)	Java Record (Java 16+)
Mutabilidad	Mutable (getters y setters)	Inmutable por diseño (campos <code>`private final`</code>)
Líneas de Código	50 - 100 líneas (boilerplate de getters, constructor, equals, hashCode)	1 línea de declaración concisa
Uso Recomendado	Entidades mutables de frameworks ORM (JPA/Hibernate)	DTOs puros de transporte en JDBC

9. Resumen Clave de la Unidad de Trabajo 2

- JDBC es la base de todo el ecosistema de persistencia en Java:** Frameworks de alto nivel como Hibernate, Spring Data o MyBatis se construyen internamente sobre la API JDBC que has aprendido en esta unidad.
- PreparedStatement es innegociable:** Separa la lógica de la consulta de los datos de entrada, garantizando seguridad absoluta frente a Inyección SQL y reutilización de planes en la SGA de Oracle.
- Las transacciones garantizan consistencia ACID:** Desactivar `setAutoCommit(false)` permite coordinar múltiples operaciones dependientes, confirmando con `commit()` o deshaciendo con `rollback()` ante fallos.
- HikariCP minimiza latencias:** El pooling de conexiones evita el elevado coste de abrir sockets TCP y negociar autenticaciones en cada petición.
- La arquitectura en capas desacopla el software:** El patrón DTO transporta datos limpios, el patrón DAO aísla las sentencias SQL y la capa de servicio gobierna la lógica transaccional del negocio.

10. Conceptos que Debes Dominar (Habilidades Comprobables)

- Configurar el driver `ojdbc11` en un proyecto Maven y conectarse a Oracle Database en localhost o contenedor Docker.
- Construir consultas SELECT y sentencias DML (INSERT, UPDATE, DELETE) seguras mediante `PreparedStatement`.
- Extraer datos de un `ResultSet` comprobando nulos con `rs.isNull()` y mapeándolos a Java Records.
- Implementar transacciones bancarias robustas con `commit()`, `rollback()` y puntos de restauración `Savepoint`.
- Llamar a procedimientos PL/SQL con parámetros de entrada y salida mediante `CallableStatement` y `registerOutParameter()`.
- Inspeccionar la estructura de tablas desconocidas en tiempo de ejecución con `ResultSetMetaData`.
- Instanciar y configurar un pool de conexiones con `HikariDataSource` gobernado por un gestor Singleton.
- Diseñar una arquitectura completa de persistencia aplicando una interfaz genérica `GenericDAO<T, ID>`.

11. Glosario Técnico y Cuestionario de Repaso

Glosario de Términos

- JDBC (Java Database Connectivity):** API del JDK que estandariza el acceso a bases de datos relacionales.
- Driver Thin de Oracle:** Controlador JDBC de Tipo 4 escrito al 100% en Java puro que se comunica directamente por sockets con Oracle sin requerir software cliente nativo (Oracle Client) instalado.

- **Soft Parse:** Reutilización inmediata por parte de Oracle de un plan de consulta ya compilado en la memoria compartida (SGA), maximizada mediante el uso de `PreparedStatement`.
- **Savepoint:** Marcador intermedio dentro de una transacción que permite deshacer operaciones posteriores sin cancelar toda la transacción previa.
- **Connection Leak:** Pérdida de conexiones en un pool provocada por no invocar `close()` tras utilizarlas, lo que agota los canales disponibles y bloquea la aplicación.
- **DAO (Data Access Object):** Patrón arquitectónico que centraliza todas las operaciones de lectura y escritura contra la base de datos para una entidad concreta.
- **DTO (Data Transfer Object):** Objeto plano sin lógica diseñado para empaquetar y transferir datos entre capas de la aplicación.

Cuestionario de Autoevaluación (10 Preguntas de Repaso)

1. ¿Cuál es la diferencia fundamental entre el JDBC Driver Manager y el Driver JDBC de un fabricante como Oracle?
2. ¿Por qué una sentencia construida con `PreparedStatement` es inmune a ataques de Inyección SQL mientras que una con `Statement` es vulnerable?
3. ¿Qué ocurre exactamente en el servidor de base de datos cuando se invoca `connection.close()` en una aplicación configurada con HikariCP frente a una conexión directa con `DriverManager`?
4. Si una columna de tipo `NUMBER` en Oracle contiene un valor `NULL`, ¿qué devolverá `rs.getInt("columna")` en Java y cómo podemos distinguir si era un cero real o un nulo?
5. ¿Cuál es el valor por defecto de `connection.getAutoCommit()` en JDBC y por qué es peligroso en operaciones que implican múltiples tablas?
6. ¿En qué casos es necesario invocar `callableStatement.registerOutParameter()` y qué consecuencia tiene omitir esta llamada antes de `cs.execute()`?
7. Explica la diferencia entre `DatabaseMetaData` y `ResultSetMetaData`, indicando un caso de uso representativo para cada interfaz.
8. ¿Qué problema en tiempo de ejecución provocaría invocar `rs.getString(1)` inmediatamente después de ejecutar `ps.executeQuery()` sin haber llamado previamente a `rs.next()`?
9. ¿Qué ventajas de legibilidad, inmutabilidad y mantenibilidad aportan los Java Records frente a los Java Beans tradicionales al modelar DTOs?
10. Describe el escenario técnico en el que resulta imprescindible utilizar un `Savepoint` dentro de una transacción compuesta.